Comparing Retrieval Strategies for Multi-Hop Question Answering: Vanilla RAG, Agentic RAG, and GraphRAG

Qian Zhou
qian.zhou@stud.uni-heidelberg.de

Abstract

Multi-hop question answering requires a system to combine evidence distributed across multiple passages, making retrieval a central challenge for retrieval-augmented generation (RAG). In this work, we study three retrieval paradigms for multi-hop question answering on HotpotQA: vanilla dense RAG, agentic iterative RAG, and graph-based RAG (GraphRAG). Vanilla RAG retrieves evidence in a single step using dense embedding similarity, whereas Agentic RAG introduces an LLM-based controller that can decide whether the current evidence is sufficient and, when necessary, generate a new query for an additional retrieval step. On a matched 200-example HotpotQA setting, Agentic RAG improves Exact Match from 0.340 to 0.600 and token-level F1 from 0.498 to 0.726 compared with vanilla RAG. It also achieves a supporting-title recall of 0.855 while requiring only 1.49 retrieval steps on average. We additionally evaluate GraphRAG on 200 hard bridge questions using a shared graph-indexed corpus, obtaining an EM of 0.480 and F1 of 0.659. Since this GraphRAG setting differs in both question subset and retrieval corpus, we treat it as an exploratory comparison rather than a strictly controlled baseline. Overall, our experiments suggest that multi-hop QA benefits not simply from more context, but from retrieval mechanisms that can identify and acquire missing evidence during reasoning.

1 Introduction

Retrieval-augmented generation (RAG) provides language models with external evidence before generating an answer, reducing the need to rely entirely on information stored in model parameters. A standard RAG pipeline typically encodes a question and a collection of candidate documents into vector representations, retrieves the most similar passages, and provides them to a language model as context. This single-stage procedure is simple and efficient, but it assumes that the original question is already a sufficiently informative retrieval query. This assumption becomes problematic for multi-hop question answering. In HotpotQA, answering a question often requires combining information from multiple documents rather than locating a single passage containing the final answer. In a typical bridge question, the first document may reveal an intermediate entity, while a second document provides the fact needed to answer the original question. The second piece of evidence may therefore be difficult to retrieve directly from the original question because the relevant bridge entity is not yet known. As a result, a retriever that performs only one similarity search may retrieve individually relevant passages while still missing part of the evidence chain required for the final answer. This work investigates how different retrieval strategies address this problem. We first implement a **Vanilla RAG** baseline using `text-embedding-3-small` representations and cosine similarity. For each HotpotQA question, the system retrieves the top-ranked candidate passages once and supplies them to GPT-4o for answer generation. We compare this system with both a direct GPT-4o baseline without retrieved context and a full-context setting in which all distractor passages associated with the question are provided to the model. We then implement an **Agentic RAG** system designed to make retrieval adaptive. Instead of assuming that one retrieval step is sufficient, an LLM-based controller examines the question together with the currently retrieved evidence and chooses between two actions: ANSWER, when the available evidence is sufficient, or SEARCH, when additional information is required. For a search action, the controller generates a new query based on both the original question and the accumulated evidence. This enables the system to use entities or facts discovered in an earlier retrieval step to target evidence required for a later reasoning step. The implementation retrieves two passages per step, allows at most

three retrieval steps, and accumulates up to six evidence documents while avoiding repeated retrieval of the same document. Our experiments indicate that this adaptive behavior is particularly useful for multi-hop QA. On the same 200 HotpotQA validation examples, vanilla RAG obtains an Exact Match (EM) score of 0.340 and an F1 score of 0.498. Providing the full question-specific context improves performance to 0.490 EM and 0.653 F1, suggesting that the single-stage retriever frequently removes information that remains useful for answering the question. Agentic RAG further improves performance to 0.600 EM and 0.726 F1. Importantly, the improvement is also reflected at the retrieval level: the agentic system achieves a supporting-title recall of 0.855, retrieves all gold supporting titles for 73.0% of the evaluated questions, and uses only 1.49 retrieval steps on average. These observations suggest that its advantage does not come merely from performing a fixed number of additional searches, but from selectively retrieving further evidence when required. Finally, we explore **GraphRAG** as a structurally different approach to multi-hop retrieval. Rather than performing retrieval independently over the distractor passages of each question, GraphRAG constructs a shared corpus from the HotpotQA contexts, extracts entities and relationships, organizes them into graph communities, and answers questions using local graph search. Our GraphRAG experiment uses 200 hard bridge questions and a shared corpus of 1,986 unique documents. It achieves 0.480 EM and 0.659 F1. However, because both the question subset and the retrieval space differ from those used for Vanilla and Agentic RAG, these scores should not be interpreted as a direct controlled comparison. Instead, the experiment provides an additional view of how explicit relational structure can be used to support multi-hop retrieval. The main goal of this study is therefore not simply to rank three RAG implementations, but to examine how different retrieval designs behave when answering questions that require distributed evidence. Our experiments provide three main observations: (1) single-shot dense retrieval can become a bottleneck even when the necessary evidence exists in the available context; (2) iterative, query-adaptive retrieval can substantially improve both supporting-evidence coverage and final answer quality; and (3) graph-structured retrieval provides an alternative mechanism for representing multi-document relationships, although a fully controlled comparison with iterative RAG requires a matched experimental setting. Overall, the results highlight that for multi-hop question answering, the key challenge is not only how much context is given to the language model, but whether the system can retrieve the right evidence at the right stage of reasoning.

2 Methods

We investigate three retrieval strategies for multi-hop question answering: single-stage dense retrieval, agent-controlled iterative retrieval, and graph-structured retrieval. The first two systems operate over the question-specific distractor passages provided by HotpotQA, while GraphRAG builds a shared index over documents collected from multiple questions. This section describes the retrieval and generation procedures used in each system.

2.1 Vanilla RAG

Our Vanilla RAG system follows a standard retrieve-then-generate pipeline. For each HotpotQA example, every candidate article is treated as one retrieval unit. The sentences belonging to the same article are concatenated, and the article title is included together with its content.

Let the candidate documents for a question be $D = \{d_1, \ldots, d_n\}$ and let q denote the question. We encode both documents and the question using `text-embedding-3-small`. The embedding of a document d_i is

$$\mathbf{d}_i = E(d_i), \tag{1}$$

and the question representation is

$$\mathbf{q} = E(q), \tag{2}$$

where $E(\cdot)$ denotes the embedding model. Retrieval scores are computed using cosine similarity,

$$s(q, d_i) = \frac{\mathbf{q}^\top \mathbf{d}_i}{\|\mathbf{q}\|_2 \|\mathbf{d}_i\|_2}. \tag{3}$$

Documents are ranked by this score, and the highest-ranked passages are concatenated as retrieved evidence. GPT-4o then receives the original question together with this evidence and is instructed to return only a short answer based on the supplied context. No query reformulation or additional retrieval is performed after this initial search. Therefore, the system relies on the original question representation to retrieve all evidence required for multi-hop reasoning.

2.2 Agentic RAG

The Agentic RAG system extends the dense retriever with an LLM-based retrieval controller. Its main purpose is to allow the retrieval query to change after intermediate evidence has been observed. This is useful for bridge questions in which an entity required for the second retrieval step becomes apparent only after reading the first retrieved passage.

For efficiency, the candidate documents for each HotpotQA question are embedded once before iterative retrieval begins. Their normalized embeddings are reused throughout all subsequent search steps. The first retrieval query is always the original question,

$$q_1 = q. \tag{4}$$

At retrieval step t, the system retrieves the two highest-scoring documents that have not already been collected. Retrieved passages are added to an accumulated evidence set E_t. Repeated retrieval of the same document is explicitly prevented, encouraging the system to acquire new evidence across steps.

After each retrieval, GPT-4o acts as a controller and examines the original question, the accumulated evidence, and the previous search queries. It selects one of two actions:

$$a_t \in \{\textsc{Search}, \textsc{Answer}\}. \tag{5}$$

If the controller selects ANSWER, it returns a short final answer and the process terminates. If it selects SEARCH, it also generates a new retrieval query,

$$q_{t+1} = C(q, E_t, H_t), \tag{6}$$

where $C(\cdot)$ denotes the LLM controller and H_t is the history of previous search queries. The newly generated query is then embedded and used for the next dense retrieval step.

This design allows later searches to explicitly target missing bridge entities, relations, dates, locations, or other information identified from earlier evidence. For example, the first retrieval may reveal the identity of a character mentioned indirectly in the original question; the next query can then search directly for an attribute of that character.

Our implementation retrieves at most two new passages per step and allows a maximum of three retrieval steps. The accumulated evidence is limited to six documents. Thus, the overall procedure can be summarized as

$$q \rightarrow \text{Retrieve} \rightarrow \text{Decide} \rightarrow \begin{cases} \text{Answer}, \\ \text{Re-query and retrieve again.} \end{cases} \tag{7}$$

If the maximum number of retrieval steps is reached, or no new document can be retrieved, GPT-4o generates a final answer using only the evidence collected so far. The controller is instructed not to use external knowledge and to return structured SEARCH or ANSWER decisions rather than a chain-of-thought explanation.

2.3 GraphRAG

GraphRAG explores a structurally different retrieval paradigm. Unlike the previous two systems, which search only within the distractor passages associated with the current question, the GraphRAG implementation first constructs a shared corpus from the contexts of 200 selected HotpotQA questions.

Documents are deduplicated by article title. If the same title occurs in multiple examples, the longest available version is retained. This process produces 1,986 unique documents in our experiment. Each document is stored with its title and article text before indexing.

We use Microsoft GraphRAG version 2.7.2 with its standard indexing pipeline. The indexing stage transforms the shared text collection into graph-oriented retrieval artifacts, including entities, relationships, communities, community reports, and vector representations. GPT-4o-mini is used as the GraphRAG language model and `text-embedding-3-small` is used for embeddings.

For answering questions, we use GraphRAG's *Local Search* method. Rather than ranking only raw passages by their similarity to the question, Local Search retrieves context associated with relevant entities and their local graph structure. Conceptually, its retrieval process can be represented as

$$q \rightarrow \text{Relevant entities} \rightarrow \text{Local graph context} \rightarrow \text{Answer}. \tag{8}$$

This provides an alternative mechanism for multi-hop QA: dependencies between pieces of evidence can be represented during indexing rather

than discovered entirely through sequential query reformulation at inference time.

The raw GraphRAG response may contain explanatory text and source references. Because HotpotQA evaluation expects concise answers, we apply a final answer-normalization step using GPT-4o-mini. The model receives the question and the raw GraphRAG response and is instructed to return only the shortest final answer without explanation or citations. We therefore evaluate the normalized answer rather than the full raw GraphRAG output.

Importantly, GraphRAG differs from Vanilla and Agentic RAG not only in retrieval mechanism but also in retrieval scope. Vanilla and Agentic RAG operate within the candidate context of each individual question, whereas GraphRAG searches a shared corpus constructed across the selected questions. We therefore treat GraphRAG as an exploratory graph-based setting rather than a strictly matched retrieval baseline.

3 Experimental Setup

3.1 Dataset

We conduct our experiments on the validation split of HotpotQA in the `distractor` setting. HotpotQA is particularly suitable for this study because many questions require combining evidence from multiple Wikipedia articles.

For Vanilla RAG and Agentic RAG, we shuffle the validation set with a fixed random seed of 42 and evaluate on 200 examples. Both systems operate on the same question set and the same question-specific candidate passages, providing a matched setting for comparing single-stage and iterative retrieval.

The GraphRAG experiment uses a different subset of the same validation split. We select 200 examples labeled as both `hard` and `bridge`, again using random seed 42. The contexts associated with these questions are merged into a shared corpus and deduplicated by article title, resulting in 1,986 unique documents. GraphRAG therefore searches a substantially larger shared retrieval space rather than the question-specific candidate set used by Vanilla and Agentic RAG. For this reason, its results are treated as an exploratory comparison rather than a directly matched baseline.

3.2 Implementation Settings

Vanilla RAG and Agentic RAG use `text-embedding-3-small` to encode questions and documents, with cosine similarity used for dense retrieval. GPT-4o is used for answer generation in both systems.

The Vanilla RAG system performs one retrieval operation using the original question and then generates an answer from the retrieved passages. We additionally evaluate two reference conditions: **Direct GPT-4o**, in which the model receives only the question, and **Full Context**, in which all candidate passages associated with a HotpotQA example are provided without retrieval.

For Agentic RAG, two new documents are retrieved at each step. The system performs at most three retrieval steps and accumulates no more than six evidence documents. Previously retrieved documents are excluded from later searches. GPT-4o serves both as the retrieval controller and as the final answer generator, with temperature set to zero.

For GraphRAG, we use Microsoft GraphRAG version 2.7.2 with the standard indexing pipeline. GPT-4o-mini is used as the language model and `text-embedding-3-small` is used for embedding generation. Questions are answered using GraphRAG's Local Search mode. Since the raw responses may contain explanations and source references, GPT-4o-mini is subsequently used to normalize each response into a short final answer before evaluation.

3.3 Evaluation Metrics

We evaluate answer quality using the standard HotpotQA answer metrics: **Exact Match (EM)** and **token-level F1**.

Exact Match assigns a score of 1 when the normalized predicted answer exactly matches the normalized gold answer, and 0 otherwise. Normalization removes differences such as capitalization, punctuation, articles, and extra whitespace.

Token-level F1 provides a softer measure by computing the overlap between tokens in the normalized prediction and gold answer. Precision and recall are defined over the shared answer tokens, and their harmonic mean gives the F1 score. This metric gives partial credit when a prediction contains the correct answer but differs slightly in wording or specificity.

We report the mean EM and F1 scores over all evaluated questions for each system.

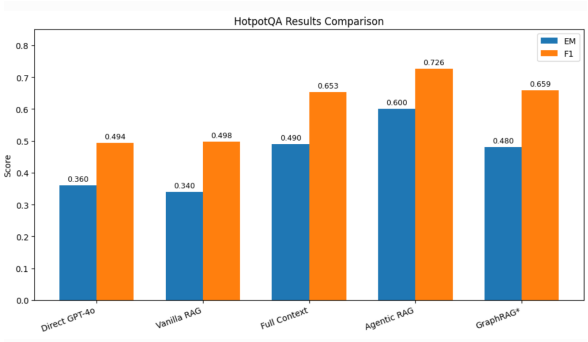


Figure 1: EM and F1 results of the evaluated systems on HotpotQA.

4 Results and Analysis

4.1 Overall Results

Figure 1 summarizes the EM and F1 results of the evaluated systems. Among the matched settings, Agentic RAG achieves the best performance, with an EM score of 0.600 and an F1 score of 0.726. Full Context also performs substantially better than Vanilla RAG, reaching 0.490 EM and 0.653 F1, while Vanilla RAG obtains 0.340 EM and 0.498 F1. The Direct GPT-4o baseline shows similar performance to Vanilla RAG.

GraphRAG achieves 0.480 EM and 0.659 F1. However, since it is evaluated on a different hard-bridge subset and uses a shared retrieval corpus, its results are reported as an exploratory comparison rather than a strictly matched baseline.

5 Limitations

This study has several limitations. First, our experiments are conducted on only 200 HotpotQA examples. Although this setting is sufficient for a small-scale course project, the results may not fully represent performance on the complete HotpotQA validation set or other multi-hop question answering datasets.

Second, the GraphRAG experiment is not directly comparable with Vanilla RAG and Agentic RAG. Vanilla and Agentic RAG are evaluated on the same randomly selected examples and retrieve from question-specific distractor passages, whereas GraphRAG is evaluated on 200 hard bridge questions and searches a shared corpus of 1,986 documents. Therefore, the GraphRAG results should be interpreted as an exploratory comparison rather than evidence that one retrieval architecture is strictly better than another.

Third, our evaluation focuses only on Exact Match and token-level F1. These automatic metrics may penalize answers that are semantically correct but differ in wording, specificity, or ordering from the gold answer. In addition, we do not perform a detailed comparison of computational cost and latency. Agentic RAG may require multiple retrieval and LLM calls, while GraphRAG introduces an additional offline graph-indexing stage. Future work could evaluate these systems under a fully matched setting and study the trade-off between answer quality and computational efficiency.

A Conclusion

In this report, we compared three retrieval strategies for multi-hop question answering on HotpotQA: Vanilla RAG, Agentic RAG, and GraphRAG. The results show that single-stage dense retrieval can become a bottleneck when the evidence required to answer a question is distributed across multiple passages. In the matched experiment, Agentic RAG achieved the strongest performance, reaching 0.600 EM and 0.726 F1, compared with 0.340 EM and 0.498 F1 for Vanilla RAG.

The comparison with the Full Context baseline further suggests that performance depends not only on the amount of available context, but also on whether the system can retrieve the relevant evidence at the appropriate stage of reasoning. By allowing the model to reformulate queries and perform additional retrieval when necessary, Agentic RAG provides a more adaptive mechanism for multi-hop evidence acquisition.

GraphRAG offers a different approach by organizing information through entities, relationships, and graph structure. Although its experimental setting is not directly matched with the other systems, it demonstrates the potential of graph-based retrieval for multi-document reasoning over a shared corpus.

Overall, our experiments suggest that retrieval design plays an important role in multi-hop question answering. Future work could evaluate all three approaches under a fully controlled setting and investigate the trade-off between retrieval quality, computational cost, and answer accuracy.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. *Retrieval-Augmented Generation for*

Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems.

[2] Zhilin Yang, Peng Qi, Saizheng Zhang, et al. 2018. *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*. Proceedings of EMNLP.

[3] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. *Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions*. Proceedings of ACL.

[4] Darren Edge, Ha Trinh, Newman Cheng, et al. 2024. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv preprint arXiv:2404.16130.